

## 개요

본 문서는 CSCI89 Lecture 8 강의 자료를 통합하여 텍스트 데이터 분석의 핵심 기법인 토픽 모델링에 대해 다룹니다. 특히, 잠재 디리클레 할당(LDA)의 기본 원리와 한계를 이해하고, 이를 확장한 구조적 토픽 모델(STM)이 메타데이터를 어떻게 활용하여 토픽 추출의 정확도를 높이는지 상세히 설명합니다. 텍스트 오토인코더의 어려움과 그 해결 방안도 함께 다루며, R 언어를 활용한 STM 모델 구축 및 해석 실습 과정을 포함합니다. 이 문서는 비전공자나 해당 주제를 처음 접하는 학습자도 완벽하게 이해할 수 있도록 쉬운 설명, 풍부한 예시, 그리고 단계별 가이드를 제공합니다.

## 용어 정리

텍스트 토픽 모델링 및 관련 개념들을 이해하기 위한 주요 용어들을 정리했습니다.

| 용어          | 쉬운 설명                                             | 원어                                      | 비고                   |
|-------------|---------------------------------------------------|-----------------------------------------|----------------------|
| 토픽 모델링      | 문서 집합에서 숨겨진 주제(토픽)를 찾아내는 통계적 방법                   | Topic Modeling                          |                      |
| 잠재 디리클레 할당  | 문서가 여러 토픽의 혼합으로 구성된다고 가정하는 토픽 모델                  | Latent Dirichlet Allocation (LDA)       | 비지도 학습               |
| 음수 행렬 분해    | 텍스트 데이터를 비음수 행렬로 분해하여 잠재 구조를 찾는 방법                | Non-negative Matrix Factorization (NMF) | 행렬 대수 기반             |
| 구조적 토픽 모델   | LDA를 확장하여 문서의 메타데이터를 토픽 모델링에 활용하는 모델              | Structural Topic Model (STM)            | 메타데이터 활용             |
| 오토인코더       | 입력 데이터를 압축(인코딩)한 후 다시 복원(디코딩)하는 신경망               | Autoencoder                             | 차원 축소, 특징 학습         |
| 잠재 변수       | 모델 내에 존재하지만 직접 관측할 수 없는 변수                        | Latent Variable                         | 예: 문서의 실제 토픽         |
| 메타데이터       | 문서 자체에 대한 추가 정보 (예: 저자, 출판일, 성별)                  | Metadata                                | 공변량(Covariate)이라고도 함 |
| 공변량         | 모델에서 결과 변수에 영향을 미칠 수 있는 보조 변수                     | Covariate                               | 메타데이터의 일종            |
| 우도          | 특정 모델과 파라미터가 관측된 데이터를 얼마나 잘 설명하는지 나타내는 값          | Likelihood                              | 모델 적합도 측정            |
| EM 알고리즘     | 잠재 변수가 있는 모델의 파라미터를 추정하는 반복적인 최적화 방법              | Expectation-Maximization (EM) Algorithm | 기대(E) 단계와 최대화(M) 단계  |
| 의미론적 일관성    | 토픽 내 단어들이 의미적으로 얼마나 관련성이 높은지 측정                   | Semantic Coherence                      | 토픽 품질 지표             |
| 배타성         | 토픽이 다른 토픽과 얼마나 겹치지 않는지 측정                         | Exclusivity                             | 토픽 품질 지표             |
| Held-out 우도 | 모델 훈련에 사용되지 않은 데이터에 대한 우도                         | Held-out Likelihood                     | 모델 일반화 성능 지표         |
| Softmax 함수  | 여러 개의 숫자(로그 확률)를 확률 분포로 변환하는 함수                   | Softmax Function                        | 합이 1이 되도록 정규화        |
| 로지스틱 정규 분포  | 정규 분포에서 샘플링한 후 Softmax를 적용하여 확률 벡터를 생성하는 분포       | Logistic Normal Distribution            | STM에서 토픽 비율 생성에 사용   |
| 어간 추출       | 단어의 접미사를 제거하여 원형(어간)으로 만드는 과정                     | Stemming                                | 텍스트 정규화              |
| 불용어         | 분석에 큰 의미가 없어 제거되는 일반적인 단어 (예: "은", "는", "이", "가") | Stop Words                              | 텍스트 전처리              |
| 하이퍼파라미터     | 모델 훈련 전에 사용자가 직접 설정해야 하는 파라미터                     | Hyperparameter                          | 예: 토픽의 수(K)          |

## 핵심 개념 및 원리

### 0.1 텍스트 오토인코더의 어려움

오토인코더는 입력 데이터를 압축(인코딩)하여 잠재 공간(latent space)에 표현한 후, 이 잠재 표현을 다시 원본 데이터로 복원(디코딩)하는 신경망 모델입니다. 이미지 처리에서는 매우 효과적이지만, 텍스트 데이터에 적용할 때는 여러 가지 고유한 어려움에 직면합니다.

#### 정의: 오토인코더의 기본 개념

오토인코더는 입력 데이터를 효율적으로 압축하고 복원하는 방법을 학습하는 신경망입니다. 인코더는 입력 데이터를 잠재 표현으로 변환하고, 디코더는 이 잠재 표현을 사용하여 원본 데이터를 재구성합니다.

#### 0.1.1 이미지 오토인코더와의 비교

이미지 데이터는 픽셀 값의 연속적인 배열로 구성되며, 공간적 인접성이 중요합니다. 컨볼루션 신경망(CNN) 기반의 오토인코더는 이러한 공간적 특징을 잘 포착하여 효율적으로 압축하고 복원할 수 있습니다. 예를 들어, 이미지의 특정 영역을 압축해도 전체적인 형태나 색상 정보는 비교적 잘 유지됩니다.

그러나 텍스트 데이터는 이산적인 단어들의 시퀀스로 구성되며, 단어의 순서와 문맥이 의미를 결정하는 데 매우 중요합니다. 텍스트를 하나의 고정된 벡터로 압축하는 과정에서 시퀀스 정보(시간적 요소)가 크게 손실될 수 있습니다.

#### 예시: 텍스트 오토인코더의 병목 현상

200 개의 단어로 이루어진 문장을 하나의 단일 벡터로 압축한다고 상상해 보세요. 이 단일 벡터는 200 개 단어의 모든 의미와 순서 정보를 담아야 합니다. 이는 마치 긴 소설의 내용을 한 문장으로 요약하는 것과 같습니다. 요약은 가능하지만, 원본의 모든 세부 정보를 완벽하게 복원하기는 매우 어렵습니다. 이처럼 정보를 압축하는 과정에서 발생하는 제약을 '병목(bottleneck)' 현상이라고 부릅니다.

#### 0.1.2 정보 손실과 재현의 어려움

텍스트 오토인코더는 전체 단어 시퀀스를 단일 벡터로 압축하는 과정에서 자연스러운 병목 현상을 만듭니다. 이 병목은 모델이 시퀀스의 핵심 정보를 추출하도록 강제하지만, 동시에 많은 세부 정보를 잃게 만듭니다.

- **시퀀스 정보 손실:** 텍스트는 단어의 순서가 중요하지만, 단일 벡터로 압축하면 이러한 시간적(sequential) 정보가 크게 손실됩니다.
- **의미론적 복잡성:** 단어 임베딩을 통해 단어를 벡터로 표현하더라도, 문장 전체의 복잡한 의미를 하나의 벡터에 담는 것은 매우 어렵습니다.

- **관측치 부족:** 충분히 많은 양의 텍스트 데이터(관측치)가 없다면, 모델은 효과적인 잠재 표현을 학습하기 어렵습니다.

### 주의: 완벽한 재현에 대한 기대

텍스트 오토인코더, 특히 순환 신경망(RNN) 기반의 모델로 문장을 완벽하게 재현하는 것은 매우 어렵습니다. 이미지 오토인코더에서 얻을 수 있는 높은 재현율을 텍스트에서 기대해서는 안 됩니다. 이 과제의 목표는 완벽한 모델을 만드는 것이 아니라, 텍스트 시퀀스 모델링의 어려움을 직접 경험하고 이해하는 것입니다.

### 0.1.3 해결 시도 및 대안

강의에서는 텍스트 오토인코더의 성능을 개선하거나 다른 방식으로 임베딩을 훈련하는 몇 가지 방법을 제시합니다.

1. **잠재 표현 차원 증가:** 잠재 벡터의 차원을 늘려 더 많은 정보를 담을 수 있도록 합니다. 하지만 이것만으로는 근본적인 문제를 해결하기 어렵습니다.
2. **Softmax 온도 함수 활용:** 디코더의 출력에서 단어를 선택할 때, 가장 높은 확률의 단어만을 선택하는 대신, Softmax 함수에 '온도(temperature)' 파라미터를 적용하여 확률 분포를 부드럽게 만들고, 그 분포에서 샘플링하여 다양한 문장을 생성할 수 있습니다. 이는 원본과 완벽히 같지는 않지만, 문법적으로 그럴듯한 문장을 생성하는 데 도움이 됩니다.
3. **데이터 증강 (Data Augmentation):** 텍스트 데이터셋이 부족할 경우, 인위적으로 데이터를 늘리는 기법입니다.
  - **동의어 대체:** 문장 내 단어를 동의어로 바꿉니다.
  - **단어 삽입/삭제/교체:** 문장에서 단어를 추가하거나, 제거하거나, 다른 단어로 교체합니다.
  - **문장 재구성:** 문장의 구조를 약간 변경합니다.

이미지 데이터 증강(회전, 확대/축소)보다 복잡하지만, 텍스트에서도 효과적일 수 있습니다.

4. **병목 없는 임베딩 훈련 (시퀀스-투-시퀀스 모델):** 오토인코더의 병목 현상 없이 임베딩 레이어를 훈련하는 방법입니다. 순환 신경망(RNN) 레이어를 여러 개 쌓아 입력 시퀀스를 출력 시퀀스로 직접 변환합니다. 이 경우, 잠재 벡터로 압축하는 과정이 없으므로 정보 손실이 적고, 높은 재현율을 얻을 수 있습니다.
  - **목표:** 임베딩 레이어 자체를 효과적으로 훈련하는 것.
  - **방법:** LSTM과 같은 순환 레이어를 사용하여 입력 시퀀스(단어 임베딩)를 출력 시퀀스(원래 문장)로 직접 매핑합니다.
  - **결과:** 거의 100%에 가까운 재현율을 얻을 수 있지만, 이는 엄밀히 말해 오토인코더의 정의(병목을 통한 압축)에 부합하지 않습니다.

**핵심 요약: 텍스트 오토인코더의 현실적 목표**

텍스트 오토인코더는 이미지 오토인코더만큼 완벽한 재현을 기대하기 어렵습니다. 특히 텍스트 데이터의 특성상 병목 현상으로 인한 정보 손실이 크기 때문입니다. 이 모델의 학습 목표는 완벽한 재현이 아니라, 모델의 한계와 텍스트 시퀀스 압축의 어려움을 이해하는 데 있습니다.

## 0.2 잠재 디리클레 할당 (Latent Dirichlet Allocation, LDA)

LDA는 문서 집합에서 추상적인 "토픽"을 발견하는 통계적 모델입니다. 각 문서는 여러 토픽의 혼합으로 구성되며, 각 토픽은 특정 단어들의 분포로 정의된다고 가정합니다.

### 정의: LDA의 핵심 가정

LDA는 모든 문서가 여러 토픽의 혼합으로 이루어져 있으며, 각 토픽은 특정 단어들의 확률 분포로 표현된다고 가정합니다. 예를 들어, "스포츠" 토픽은 "야구", "축구", "경기"와 같은 단어들이 높은 확률로 나타날 것입니다.

### 0.2.1 LDA의 동작 원리

LDA는 다음과 같은 생성 과정을 가정하여 문서가 생성되었다고 봅니다. 이 가정을 바탕으로 역으로 토픽과 문서-토픽 분포를 추정합니다.

1. **문서별 토픽 분포 선택:** 각 문서  $d$ 에 대해, 디리클레 분포(Dirichlet distribution)에서 토픽 분포  $\theta_d$ 를 샘플링합니다.  $\theta_d$ 는 문서  $d$ 가 각 토픽에 할당될 확률을 나타내는 벡터입니다 (예: 토픽 1에 60%, 토픽 2에 40%).
2. **단어별 토픽 선택:** 문서  $d$ 의 각 단어 위치에 대해,  $\theta_d$ 에 따라 토픽  $z$ 를 선택합니다.
3. **단어 생성:** 선택된 토픽  $z$ 에 해당하는 단어 분포  $\phi_z$ 에서 실제 단어  $w$ 를 샘플링합니다.

이 과정에서  $\alpha$ 와  $\beta$ 는 디리클레 분포의 하이퍼파라미터로, 각각 문서-토픽 분포와 토픽-단어 분포의 희소성(sparsity)을 조절합니다.

### 예시: LDA의 토픽 할당 예시

뉴스 기사를 읽을 때, 하나의 기사가 정치와 경제를 동시에 다룰 수 있습니다. LDA는 이 기사를 "정치 토픽 70% + 경제 토픽 30%"와 같이 여러 토픽의 혼합으로 해석합니다. 각 토픽은 다시 "정치" 토픽은 '선거', '국회', '법안' 등의 단어와, "경제" 토픽은 '주식', '환율', '성장률' 등의 단어와 높은 연관성을 가집니다.

### 0.2.2 LDA의 특징

- **비지도 학습 (Unsupervised Learning):** LDA는 문서에 미리 레이블이 지정되어 있지 않아도 토픽을 발견할 수 있습니다. 이는 대규모 텍스트 데이터에서 유용합니다.
- **문서-토픽 혼합:** 각 문서는 단일 토픽이 아닌 여러 토픽의 혼합으로 표현됩니다.
- **토픽-단어 분포:** 각 토픽은 특정 단어들의 확률 분포로 정의됩니다.
- **잠재 변수:** 어떤 토픽이 선택되어 특정 단어가 생성되었는지 직접 관측할 수 없으므로, 토픽 할당  $Z$ 는 잠재 변수(latent variable)로 간주됩니다.

### 0.2.3 LDA의 한계: 최적 토픽 수 결정

LDA의 가장 큰 도전 과제 중 하나는 최적의 토픽 수  $K$ 를 결정하는 것입니다.  $K$ 는 모델 훈련 전에 사용자가 직접 설정해야 하는 하이퍼파라미터입니다.

- **너무 많은 토픽:** 유사한 아이디어를 가진 여러 토픽이 생성되어 토픽 간 구분이 모호해질 수 있습니다 (예: '고양이 1' 토픽과 '고양이 2' 토픽).
- **너무 적은 토픽:** 서로 다른 아이디어가 하나의 토픽으로 묶여 토픽의 의미가 불분명해질 수 있습니다 (예: '컴퓨터'와 '고양이'가 하나의 토픽으로 묶이는 경우).

최적의  $K$ 를 찾기 위해 다음과 같은 지표들을 활용합니다.

- **의미론적 일관성 (Semantic Coherence):** 토픽 내 단어들이 얼마나 의미적으로 관련성이 높은지 측정합니다.
- **배타성 (Exclusivity):** 토픽이 다른 토픽과 얼마나 겹치지 않는지 측정합니다.
- **Held-out 우도 (Held-out Likelihood):** 훈련에 사용되지 않은 데이터에 대한 모델의 우도를 측정하여 일반화 성능을 평가합니다.

일반적으로 여러  $K$  값에 대해 LDA를 실행하고, 이 지표들을 시각화하여 최적의  $K$ 를 선택합니다. 이는 종종 수동적인 작업이 될 수 있습니다.

#### 주의: LDA의 확률적 특성

LDA는 확률적(stochastic) 알고리즘이므로, 같은 데이터와 파라미터로 여러 번 실행해도 항상 동일한 결과를 얻지 못할 수 있습니다. 토픽의 순서가 바뀌거나, 토픽을 구성하는 단어의 조합이 미묘하게 달라질 수 있습니다. 이는 우도 함수의 최대값이 유일하지 않거나, 여러 지역 최대값(local maxima)을 가질 수 있기 때문입니다. 따라서 실제 적용 시에는 여러 번 실행하여 가장 좋은 결과를 선택하는 것이 일반적입니다.

### 0.3 음수 행렬 분해 (Non-negative Matrix Factorization, NMF)

NMF는 텍스트 데이터를 나타내는 행렬을 두 개의 작은 비음수(non-negative) 행렬로 분해하는 방법입니다. 이 분해된 행렬들은 텍스트의 잠재적인 "토픽" 구조를 드러내는 데 사용될 수 있습니다.

#### 정의: NMF의 기본 개념

NMF는 모든 원소가 음수가 아닌 행렬  $V$ 를 두 개의 비음수 행렬  $W$ 와  $H$ 의 곱으로 근사하는 방법입니다 ( $V \approx WH$ ). 여기서  $V$ 는 문서-단어 행렬,  $W$ 는 문서-토픽 행렬,  $H$ 는 토픽-단어 행렬로 해석될 수 있습니다.

#### 0.3.1 NMF의 동작 원리

- 문서-단어 행렬 ( $V$ ): 각 행은 문서를, 각 열은 어휘의 단어를 나타냅니다. 셀 값은 해당 단어가 문서에 나타나는 빈도 등을 나타냅니다.
- 문서-토픽 행렬 ( $W$ ): 각 행은 문서를, 각 열은 "토픽"을 나타냅니다. 각 셀은 문서가 특정 토픽에 얼마나 기여하는지를 나타냅니다.
- 토픽-단어 행렬 ( $H$ ): 각 행은 "토픽"을, 각 열은 어휘의 단어를 나타냅니다. 각 셀은 특정 토픽이 특정 단어에 얼마나 기여하는지를 나타냅니다.

NMF는  $W$ 와  $H$ 의 차원(토픽 수)을 원본 행렬  $V$ 의 차원보다 작게 설정하여 차원 축소 효과를 얻습니다. 이 과정에서 함께 자주 나타나는 단어들이 하나의 토픽으로 묶이는 경향이 있습니다.

#### 예시: NMF 행렬 분해 예시

어휘에 6개의 단어(고양이, 개, 먹이, 짖다, 야옹, 털)가 있고, 3개의 문서가 있다고 가정해 봅시다. 원본 문서-단어 행렬  $V$  (3x6)는 다음과 같습니다:

- 문서 1: 고양이, 야옹, 털
- 문서 2: 개, 짖다, 먹이
- 문서 3: 고양이, 개, 털, 먹이

NMF는 이  $V$ 를 두 개의 "토픽"을 가진  $W$  (3x2)와  $H$  (2x6)로 분해할 수 있습니다.

- $W$ 의 첫 번째 열은 문서가 '고양이 토픽'에 얼마나 기여하는지, 두 번째 열은 '개 토픽'에 얼마나 기여하는지를 나타낼 수 있습니다.
- $H$ 의 첫 번째 행은 '고양이 토픽'이 어떤 단어들(고양이, 야옹, 털)로 구성되는지, 두 번째 행은 '개 토픽'이 어떤 단어들(개, 짖다, 먹이)로 구성되는지를 나타낼 수 있습니다.



### 0.3.2 LDA와 NMF의 비교

LDA와 NMF는 모두 텍스트에서 잠재적인 토픽을 추출하는 데 사용되지만, 근본적인 접근 방식에서 차이가 있습니다.

| 특징    | LDA (잠재 디리클레 할당)             | NMF (음수 행렬 분해)          |
|-------|------------------------------|-------------------------|
| 기본 이론 | 확률론적 모델 (베이지스 추론)            | 행렬 대수 (선형 대수)           |
| 목표    | 관측된 데이터의 우도 (likelihood) 최대화 | 원본 행렬을 두 비음수 행렬의 곱으로 근사 |
| 토픽 정의 | 단어들의 확률 분포                   | 단어들의 가중치 벡터             |
| 문서 정의 | 토픽들의 확률 분포                   | 토픽들의 가중치 벡터             |
| 결과 특성 | 확률적, 실행마다 결과가 달라질 수 있음       | 결정론적 (일반적으로), 재현 가능     |
| 해석    | 문서-토픽 분포, 토픽-단어 분포           | 문서-토픽 행렬, 토픽-단어 행렬      |
| 주요 용도 | 토픽 발견, 문서 분류, 추천 시스템         | 차원 축소, 특징 추출, 이미지 처리    |

## 0.4 최대 우도 추정 (Maximum Likelihood Estimation, MLE) 및 EM 알고리즘

LDA와 같은 확률론적 모델에서 파라미터를 추정하는 핵심 방법은 최대 우도 추정입니다. 특히 모델에 직접 관측할 수 없는 잠재 변수(latent variable)가 포함되어 있을 때, EM(Expectation-Maximization) 알고리즘이 사용됩니다.

### 정의: 최대 우도 추정 (MLE)

MLE는 관측된 데이터가 주어졌을 때, 이 데이터를 생성했을 가능성이 가장 높은(우도가 최대인) 모델 파라미터를 찾는 방법입니다.

#### 0.4.1 MLE의 직관적 이해

##### 예시: 젖은 고양이 비유

집에 돌아왔는데 고양이가 젖어 있습니다. 이 관측 데이터( $D$ : 고양이가 젖었다)를 바탕으로 무엇이 일어났을지 추론해야 합니다.

- 가설 1: 밖에 비가 왔다 ( $H_1$ : 비가 옴).
- 가설 2: 누군가 고양이에게 물을 뿌렸다 ( $H_2$ : 물 뿌림).

MLE는 각 가설 하에서 고양이가 젖을 확률( $P(D|H_1)$  또는  $P(D|H_2)$ )을 계산하고, 이 확률(우도)이 가장 높은 가설을 선택합니다. 만약 비가 왔을 때 고양이가 젖을 확률이 100%에 가깝고, 누군가 물을 뿌릴 확률이 낮다면, "비가 왔다"는 가설이 더 높은 우도를 가집니다.

이처럼 MLE는 관측된 데이터가 특정 모델 파라미터 하에서 발생할 확률을 최대화하는 파라미터를 찾습니다.

#### 0.4.2 EM 알고리즘: 잠재 변수가 있는 경우

LDA와 같이 토픽 할당( $Z$ )과 같은 잠재 변수가 모델에 포함되어 직접 관측할 수 없을 때, 일반적인 MLE는 적용하기 어렵습니다. 이때 EM 알고리즘이 사용됩니다.

### 정의: EM 알고리즘

EM 알고리즘은 잠재 변수가 포함된 통계 모델의 파라미터를 추정하기 위한 반복적인 방법입니다. 기대(Expectation) 단계와 최대화(Maximization) 단계로 구성됩니다.

1. **E-단계 (기대 단계, Expectation Step):** 현재 추정된 모델 파라미터를 사용하여 잠재 변수의 기대값(또는 잠재 변수에 대한 우도의 기대값)을 계산합니다. 즉, 관측할 수 없는 잠재 변수가 어떤 값을 가질지 '기대'합니다.
2. **M-단계 (최대화 단계, Maximization Step):** E-단계에서 계산된 잠재 변수의 기대값을 마치 관측된 데이터처럼 사용하여, 모델 파라미터의 우도를 최대화합니다. 새로운 파라미터

추정치를 얻습니다.

이 두 단계는 파라미터가 수렴할 때까지 반복됩니다. LDA에서는 이 EM 알고리즘을 사용하여 문서-토픽 분포( $\theta$ )와 토픽-단어 분포( $\phi$ )를 추정합니다.

#### 주의: LDA의 비고유한 해

LDA는 EM 알고리즘을 사용하며, 우도 함수가 여러 지역 최대값(local maxima)을 가질 수 있습니다. 이로 인해 알고리즘이 시작되는 초기값에 따라 다른 결과(다른 토픽 조합이나 순서)에 수렴할 수 있습니다. 따라서 LDA를 여러 번 실행하고, 의미론적 일관성이나 배타성 같은 지표를 사용하여 가장 좋은 결과를 선택하는 것이 중요합니다.

## 구조적 토픽 모델링 (Structural Topic Model, STM)

### 0.5 STM의 필요성: 메타데이터 활용

LDA의 주요 한계 중 하나는 문서 자체의 내용만을 사용하여 토픽을 추출한다는 점입니다. 하지만 실제 문서에는 저자, 출판일, 출판 매체, 문서 유형 등 다양한 '메타데이터'가 존재하며, 이러한 정보는 토픽의 특성이나 분포에 큰 영향을 미칠 수 있습니다. STM은 이러한 메타데이터를 모델에 통합하여 토픽 모델링의 정확도와 해석 가능성을 높입니다.

#### 정의: 메타데이터 (Metadata) 또는 공변량 (Covariate)

메타데이터는 문서의 내용 외에 문서에 대한 추가적인 정보를 의미합니다. 예를 들어, 뉴스 기사의 경우 저자의 성별, 출판 연도, 출판 매체 등이 메타데이터가 될 수 있습니다. STM에서는 이러한 메타데이터를 '공변량(covariate)'이라고 부르며, 토픽 분포에 영향을 미치는 변수로 활용합니다.

#### 예시: 메타데이터의 중요성

학생 강의 평가 데이터를 분석한다고 가정해 봅시다.

- **문서 내용:** 학생들이 작성한 평가 텍스트.
- **메타데이터:** 강사의 성별, 학과, 강의 연도, 학생의 학년 등.

만약 "강사의 열정"에 대한 토픽이 있다면, 이 토픽의 출현 빈도(prevalence)가 강사의 성별이나 학과에 따라 달라질 수 있습니다. LDA는 이러한 관계를 파악할 수 없지만, STM은 메타데이터를 활용하여 "여성 강사에 대한 평가에서 '열정' 토픽이 더 자주 언급된다"와 같은 통찰을 얻을 수 있습니다.

### 0.6 STM의 원리 및 공식

STM은 LDA의 토픽 생성 과정 중, 문서별 토픽 분포  $\theta_d$ 를 생성하는 부분에 메타데이터를 통합합니다.

#### 0.6.1 토픽 비율( $\theta_d$ ) 생성의 변화

LDA에서는  $\theta_d$ 가 디리클레 분포에서 직접 샘플링되었습니다. STM에서는  $\theta_d$ 가 '로지스틱 정규 분포(Logistic Normal Distribution)'에서 생성됩니다. 이 분포의 중심(평균)은 메타데이터와 선형적으로 연결됩니다.

$$\theta_d \sim \text{Logistic Normal}(\mu_d, \Sigma)$$

여기서  $\mu_d$ 는 문서  $d$ 의 메타데이터  $X_d$ 와 계수  $\gamma$ 의 선형 결합으로 정의됩니다.

$$\mu_d = X_d \gamma$$

- $X_d$ : 문서  $d$ 에 해당하는 메타데이터 벡터 (예: [1, 강사성별, 강의연도]).
- $\gamma$ : 메타데이터 변수들이 토픽 분포에 미치는 영향을 나타내는 계수 벡터.
- $\Sigma$ : 토픽 간의 공분산을 나타내는 행렬. 이는 토픽들이 서로 독립적이지 않고 상관관계를 가질 수 있음을 의미하며, 모델 훈련 과정에서 추정됩니다.

#### 정의: 로지스틱 정규 분포 (Logistic Normal Distribution)

로지스틱 정규 분포는 다변량 정규 분포에서 벡터를 샘플링한 후, 이 벡터에 Softmax 함수를 적용하여 확률 벡터(합이 1이 되는 벡터)를 생성하는 분포입니다. STM에서는 이 방법을 통해 메타데이터에 따라 토픽 비율  $\theta_d$ 를 생성합니다.

#### 예시: $\mu_d = X_d \gamma$ 의 의미

만약 메타데이터  $X_d$ 가 강사의 성별(0: 남성, 1: 여성)과 강의 연도( $Y$ )로 구성된다면,  $\mu_d$ 는 다음과 같이 표현될 수 있습니다.

$$\mu_d = \gamma_0 + \gamma_1 \cdot \text{성별}_d + \gamma_2 \cdot Y_d$$

이것은 마치 선형 회귀 모델처럼, 강사의 성별과 강의 연도가 문서  $d$ 의 토픽 분포 중심  $\mu_d$ 에 어떻게 영향을 미치는지를 나타냅니다. 이  $\mu_d$ 를 중심으로 하는 정규 분포에서 샘플링한 후 Softmax를 적용하여 최종 토픽 비율  $\theta_d$ 를 얻습니다.

## 0.7 STM의 장점 및 활용

STM은 메타데이터를 통합함으로써 LDA의 한계를 극복하고 다음과 같은 장점을 제공합니다.

1. **더 정확한 토픽 할당**: 메타데이터를 고려하여 문서에 토픽을 할당하므로, 토픽 추출의 정확도가 높아집니다.
2. **가설 검정 및 통찰**: 메타데이터 변수가 특정 토픽의 출현 빈도(prevalence)에 유의미한 영향을 미치는지 통계적으로 검정할 수 있습니다.
  - **예시 1: 시간 경과에 따른 토픽 변화**: 특정 토픽(예: '경제 위기')의 언급 빈도가 시간이 지남에 따라 어떻게 변하는지 분석할 수 있습니다.
  - **예시 2: 강사 성별에 따른 평가 편향**: "강사의 열정" 토픽이 여성 강사 평가에서 더 자주 언급되는지, 남성 강사 평가에서 "강의의 흥미로움" 토픽이 더 자주 언급되는지 등을 분석하여 잠재적인 편향을 탐지할 수 있습니다.
3. **시각화**: 메타데이터 변수에 따른 토픽 prevalence의 변화를 그래프로 쉽게 시각화할 수 있습니다.

**핵심 요약: STM의 핵심 가치**

STM은 단순히 토픽을 추출하는 것을 넘어, 토픽과 문서의 메타데이터 간의 관계를 탐색하고 통계적으로 검정할 수 있게 함으로써 사회과학, 경제학 등 다양한 분야에서 깊이 있는 통찰을 제공합니다.

## STM 실습: R 언어 활용

이 섹션에서는 R 언어와 ‘stm’ 패키지를 사용하여 구조적 토픽 모델을 구축하고 분석하는 과정을 단계별로 설명합니다.

### 0.8 R 환경 설정 및 데이터 준비

‘stm’ 패키지는 R에서 가장 널리 사용되는 구조적 토픽 모델링 도구입니다.

#### 정의: R Markdown (RMD)

R Markdown은 코드, 텍스트, 그리고 코드 실행 결과를 하나의 문서에 통합할 수 있는 도구입니다. Jupyter Notebook과 유사하게 코드 블록을 실행하고 그 결과를 문서에 포함시킬 수 있습니다. 코드 블록은 세 개의 백틱(‘ ` ’)으로 둘러싸여 구분됩니다.

#### 0.8.1 ‘stm’ 패키지 설치 및 로드

R에서 ‘stm’ 패키지를 사용하려면 먼저 설치하고 로드해야 합니다.

```
1 # stm 패키지 설치 최초 ( 회만1 실행 )
2 install.packages("stm")
3
4 # stm 패키지 로드 새로운 ( R 세션 시작 시마다 실행 )
5 library(stm)
6 # dplyr, ggplot2 등 다른 필요한 패키지도 로드
7 library(dplyr)
8 library(ggplot2)
```

Listing 1: ‘stm’ 패키지 설치 및 로드

#### 0.8.2 데이터셋 준비

실습에서는 Kaggle의 "Million News Headlines" 데이터셋을 사용합니다. 이 데이터셋은 뉴스 헤드라인 텍스트와 출판일(publish date)을 포함합니다.

```
1 # 데이터셋 로드 (CSV 파일 경로 지정)
2 # 데이터는 의 Canvas 'data' 폴더 또는 에서 Kaggle 다운로드 가능
3 news_data <- read.csv("abcnews-date-text.csv", stringsAsFactors = FALSE)
4
5 # 데이터 차원 확인 백만 ( 개 이상의 헤드라인 )
6 dim(news_data)
7
8 # 데이터 구조 확인
9 str(news_data)
```

Listing 2: 데이터셋 로드 및 구조 확인

`'dim(news_data)'` `'[1000000, 2]'` 2 `'str(news_data)'` `'publish_date'` `'headline_text'` .

### 0.8.3 날짜 변수 처리

`'publish_date'` `'YYYYMMDD'` . STM .

```
1 # publish_를 date Date 형식으로 변환
2 news_data$date_obj <- as.Date(as.character(news_data$publish_date), format="%Y%m%d")
3
4 # 연도와 월 추출
5 news_data$year <- as.numeric(format(news_data$date_obj, "%Y"))
6 news_data$month <- as.numeric(format(news_data$date_obj, "%m"))
7
8 # 연속적인 숫자형 날짜 변수 생성 연도( + 월(-1)/12)
9 # 예: 년2003 월은2 2003 + (2-1)/12 = 2003.083...
10 news_data$numeric_date <- news_data$year + (news_data$month - 1) / 12
11
12 # 첫 몇 개 값 확인
13 head(news_data$numeric_date)
```

Listing 3: 날짜 변수 전처리

이렇게 생성된 `'numeric_date'` .

### 0.8.4 데이터 샘플링

백만 개 이상의 데이터를 모두 처리하는 데는 시간이 오래 걸리므로, 개발 단계에서는 데이터의 일부만 샘플링하여 사용합니다.

```
1 # 전체 데이터의 만5% 샘플링하여 사용
2 # 실제 분석 시에는 이 줄을 주석 처리하고 전체 데이터를 사용
3 news_sample <- news_data %>% sample_frac(0.05)
4
5 # 샘플링된 데이터의 차원 확인
6 dim(news_sample)
```

Listing 4: 데이터 샘플링

## 0.9 텍스트 전처리

STM 모델을 훈련하기 전에 텍스트 데이터를 전처리해야 합니다. `'stm'` 패키지는 `'textProcessor'`와 `'prepareDocuments'` 함수를 제공하여 이 과정을 자동화합니다.

```
1 # 문서 텍스트와 메타데이터 분리
2 docs <- news_sample$headline_text
3 meta <- data.frame(date = news_sample$numeric_date) # 메타데이터는 데이터프레임 형식으
  로
4
```



```

5 # 텍스트 전처리: 소문자화, 불용어 제거, 숫자 제거, 구두점 제거, 어간 추출
6 processed <- textProcessor(
7   documents = docs,
8   metadata = meta,
9   lowercase = TRUE,
10  removestopwords = TRUE,
11  removenumbers = TRUE,
12  removepunctuation = TRUE,
13  stem = TRUE
14 )
15
16 # 전처리 결과 요약
17 summary(processed)

```

Listing 5: 텍스트 전처리 ('textProcessor')

'textProcessor'의 'summary()' 출력은 제거된 단어, 빈 문서 수, 최종 어휘 크기 등을 보여줍니다. 예를 들어, "removing 9000 observations from 21000 due to frequency"는 빈번하지 않은 단어들 때문에 9000개의 단어가 어휘에서 제거되었음을 의미합니다. "50 documents become empty"는 전처리 후 50개의 문서가 비어있게 되어 제거되었음을 의미합니다. 이 경우, 해당 문서에 대응하는 메타데이터도 함께 제거됩니다.

```

1 # 추가적인 문서 준비 회귀 (단어 제거 등)
2 out <- prepareDocuments(
3   documents = processed$documents,
4   vocab = processed$vocab,
5   metadata = processed$meta,
6   lower.thresh = 10 # 최소 빈 10 이상 등장하는 단어만 유지
7 )
8
9 # 준비된 문서, 어휘, 메타데이터 추출
10 docs_final <- out$documents
11 vocab_final <- out$vocab
12 meta_final <- out$meta # 전처리 과정에서 변경될 수 있으므로 새로 추출

```

Listing 6: 문서 준비 ('prepareDocuments')

'prepareDocuments'는 'lower.thresh'와 같은 파라미터를 통해 회귀 단어를 추가로 제거할 수 있습니다. 이 과정에서 문서나 어휘가 더 줄어들 수 있으며, 'meta\_final' 'docs\_final' .

## 0.10 STM 모델 실행 및 결과 해석

이제 전처리된 데이터를 사용하여 STM 모델을 훈련하고 결과를 해석합니다.

### 0.10.1 STM 모델 훈련

'stm()' 함수를 사용하여 모델을 훈련합니다. 'K'는 토픽의 수, 'formula'는 공변량을 지정합니다.

```

1 # 재현 가능한 결과를 위해 시드 설정
2 set.seed(89)
3
4 # STM 모델 훈련 (K로=5 가정)
5 # formula = ~ 는 date 'date' 변수를 공변량으로 사용하겠다는 의미
6 stm_model <- stm(
7   documents = docs_final,
8   vocab = vocab_final,
9   K = 5, # 토픽 수 여기서는 (개로) 5 가정)
10  formula = ~ date, # 공변량 지정
11  data = meta_final, # 메타데이터
12  init.type = "Spectral", # 초기화 방식
13  max.em.iter = 100 # EM 알고리즘 최대 반복 횟수
14 )

```

Listing 7: STM 모델 훈련

‘stm()’ 함수는 EM 알고리즘을 사용하여 모델 파라미터를 추정합니다. ‘E-step’과 ‘M-step’이 반복적으로 수행되며, 수렴할 때까지 또는 ‘max.em.iter’에 도달할 때까지 진행됩니다.

### 0.10.2 모델 결과 요약

훈련된 모델의 ‘summary()’를 통해 각 토픽과 가장 연관된 단어들을 확인할 수 있습니다.

```

1 summary(stm_model)

```

Listing 8: STM 모델 요약

‘summary()’ 출력은 각 토픽에 대해 가장 높은 확률을 가진 단어들을 보여줍니다. 하지만 이 단어 들만으로는 토픽의 의미를 명확히 파악하기 어려울 수 있습니다 (예: ‘Australia’와 같은 단어는 뉴스 헤드라인 데이터셋에서 흔히 나타나므로 특정 토픽을 잘 대표하지 못할 수 있습니다).

### 0.10.3 토픽 prevalence 시각화

‘plot()’ 함수는 각 토픽의 평균 출현 빈도(prevalence)를 시각화하여, 어떤 토픽이 전체 문서 집합에서 가장 지배적인지 보여줍니다.

```

1 plot(stm_model, type = "summary", n = 5) # 개5 토픽 모두 표시

```

Listing 9: 토픽 prevalence 시각화

이 그래프는 각 토픽이 전체 문서에서 차지하는 평균 비율을 보여줍니다. 예를 들어, ‘Topic 2’가 30%로 가장 높다면, 이 토픽이 전체 뉴스 헤드라인에서 가장 자주 다루어지는 주제임을 의미합니다.

### 0.10.4 워드 클라우드 시각화

‘cloud()’ 함수는 각 토픽에 대한 워드 클라우드를 생성합니다.

```
1 cloud(stm_model, topic = 1, scale = c(2.2, .2)) # Topic 에 1 대한 워드 클라우드
```

Listing 10: 토픽별 워드 클라우드

워드 클라우드는 토픽과 연관된 단어들을 시각적으로 보여주지만, ‘summary()’와 마찬가지로 토픽의 의미를 명확히 해석하는 데 한계가 있을 수 있습니다.

## 0.11 토픽 효과 추정 및 시각화

STM의 강력한 기능 중 하나는 공변량(메타데이터)이 토픽의 prevalence에 미치는 영향을 추정하고 시각화하는 것입니다.

### 0.11.1 ‘estimateEffect()’ 함수

‘estimateEffect()’ 함수는 훈련된 STM 모델의 파라미터를 사용하여, 공변량의 변화에 따른 토픽 prevalence의 변화를 추정합니다.

```
1 # 번부터 1 번까지 5 모든 토픽에 대해 효과 추정
2 # formula = ~ 는 date 'date' 변수가 토픽 prevalence 미치는 영향을 분석하겠다는 의미
3 effect <- estimateEffect(
4   1:5 ~ date, # 모든 토픽 (1:5)에 대해 date 변수의 효과 추정
5   stmobj = stm_model, # 훈련된 STM 모델 객체
6   metadata = meta_final, # 메타데이터
7   uncertainty = "Global" # 불확실성 추정 방식
8 )
9
10 # 추정된 효과 객체 요약
11 summary(effect)
```

Listing 11: 토픽 효과 추정

‘estimateEffect()’는 추정된 파라미터(사후 분포)를 기반으로 토픽 prevalence의 기대값을 반복적으로 샘플링하고, 이 샘플링된 값들을 공변량에 대해 회귀 분석하여 효과를 추정합니다.

### 0.11.2 토픽 prevalence 변화 시각화

‘plot()’ 함수를 ‘estimateEffect()’의 결과와 함께 사용하여 공변량에 따른 토픽 prevalence의 변화를 시각화할 수 있습니다.

```
1 plot(
2   effect,
3   covariate = "date", # 공변량 지정
4   topics = 1:5, # 모든 토픽 표시
```

```

5  model = stm_model, # STM 모델 객체
6  method = "continuous", # 공변량이 연속형 변수임을 지정
7  xlab = "Year", # 축 x 레이블
8  main = "Topic Prevalence Over Time", # 그래프 제목
9  linecol = "blue", # 추세선 색상
10 printlegend = TRUE # 범례 표시
11 )

```

Listing 12: 토픽 prevalence 변화 시각화

이 그래프는 `numeric_date` ( ) `prevalence` . , `prevalence` .

#### 주의: 선형 추세 가정의 한계

‘method = "continuous"’를 사용할 경우, 공변량과 토픽 prevalence 간의 관계를 선형으로 가정합니다. 만약 비선형적인 관계(예: U자형, 주기적 변화)가 예상된다면, ‘formula’에 ‘date<sup>2</sup>’, ‘date’ `prevalence` .

## 0.12 최적 토픽 수(K) 탐색

STM 모델에서 가장 중요한 하이퍼파라미터 중 하나는 토픽의 수  $K$ 입니다. 최적의  $K$ 를 찾기 위해 ‘searchK()’ 함수를 사용하여 여러  $K$  값에 대해 모델을 실행하고, 다양한 평가 지표를 비교합니다.

### 0.12.1 ‘searchK()’ 함수 실행

‘searchK()’는 지정된 범위의  $K$  값에 대해 STM 모델을 훈련하고, 각  $K$ 에 대한 평가 지표를 계산합니다.

```

1  # 개부터 2 개까지의 10 토픽 수에 대해 모델 실행
2  k_results <- searchK(
3    documents = docs_final,
4    vocab = vocab_final,
5    K = c(2:10), # 탐색할 토픽 수 범위
6    formula = ~ date,
7    data = meta_final,
8    init.type = "Spectral",
9    max.em.iter = 50 # 탐색이므로 반복 횟수를 줄일 수 있음
10 )
11
12 # 결과 요약
13 plot(k_results)

```

Listing 13: 최적 토픽 수 탐색 (‘searchK()’)

‘plot(k\_results)’ `held-out likelihood`, `(residuals)` , `(semantic coherence)` `(exclusivity)` .

### 0.12.2 평가 지표 시각화 및 최적 $K$ 선택

‘searchK()’ 결과에서 의미론적 일관성과 배타성 값을 추출하여 시각화하고, 이 두 지표를 동시에 최대화하는  $K$ 를 선택합니다.

```

1 # 각 예K 대한 의미론적 일관성(coherence)과 배타성(exclusivity) 추출
2 coherence_values <- unlist(k_results$results$semcoh)
3 exclusivity_values <- unlist(k_results$results$exclus)
4 k_values <- k_results$results$K
5
6 # 데이터프레임 생성
7 metrics_df <- data.frame(
8   K = k_values,
9   Coherence = coherence_values,
10  Exclusivity = exclusivity_values
11 )
12
13 # 를ggplot2 이용한 시각화
14 ggplot(metrics_df, aes(x = Exclusivity, y = Coherence, label = K)) +
15   geom_point(size = 3) +
16   geom_text(vjust = -1, hjust = 0.5) +
17   labs(title = "Coherence vs. Exclusivity for Different K",
18        x = "Exclusivity",
19        y = "Coherence") +
20   theme_minimal()

```

Listing 14: 일관성 및 배타성 시각화

이 그래프에서 우측 상단(높은 일관성과 높은 배타성)에 위치하는  $K$  값을 선택하는 것이 일반적입니다. 두 지표의 스케일이 다를 수 있으므로, 각 지표를 0-1 사이로 재조정(rescale)한 후 평균을 내어 최적  $K$ 를 선택하는 방법도 있습니다.

#### 주의: 최적 $K$ 선택의 주관성

최적의  $K$ 를 선택하는 것은 완전히 객관적이지 않을 수 있습니다. 통계적 지표 외에도, 추출된 토픽들이 인간이 이해하기에 얼마나 의미 있고 해석 가능한지(interpretability)도 중요한 고려 사항입니다. ‘findThoughts()’ 함수를 사용하여 각  $K$ 에서 추출된 토픽들의 대표 문서를 확인하며 의미를 평가하는 것이 좋습니다.

### 0.12.3 최적 모델 선택 및 재현성

‘searchK()’는 각  $K$  값에 대해 여러 번 실행될 수 있으며, 각 실행마다 결과가 다를 수 있습니다. 따라서 특정  $K$ 에 대해 가장 좋은 성능을 보인 모델을 명시적으로 선택하는 것이 중요합니다.

```

1 # 예를 들어, K가=5 최적이라고 판단했을 때, 해당 예K 대한 최적 모델을 선택
2 # k_results$runout[[K_index]][[run_index]] 형태로 접근 가능
3 # 여기서는 K=5 해당하는 첫 번째 실행 모델을 선택한다고 가정
4 selected_k_model <- k_results$runout[[which(k_results$results$K == 5)]][[1]]

```

```

5
6 # 선택된 모델 요약
7 summary(selected_k_model)

```

Listing 15: 최적 K에 대한 모델 선택

이렇게 선택된 모델을 사용하여 이후의 분석을 진행합니다.

### 0.13 토픽 내용 이해

토픽의 의미를 명확히 이해하기 위해서는 ‘findThoughts()’ 함수를 사용하여 각 토픽과 가장 연관성이 높은 문서들을 살펴보는 것이 가장 효과적입니다.

```

1 # Topic 과1 가장 연관된 문서 개3 확인
2 findThoughts(
3     model = stm_model,
4     texts = docs, # 원본 문서 텍스트 전처리 ( 전 )
5     n = 3, # 표시할 문서 수
6     topics = 1 # 확인할 토픽 번호
7 )
8
9 # Topic 과3 가장 연관된 문서 개3 확인
10 findThoughts(
11     model = stm_model,
12     texts = docs,
13     n = 3,
14     topics = 3
15 )

```

Listing 16: 토픽별 대표 문서 확인 (‘findThoughts’)

‘findThoughts()’는 특정 토픽에 대한 ‘theta’ 값이 가장 높은 문서들을 보여줍니다. 이 문서들을 읽어보면 해당 토픽이 어떤 주제를 다루는지 직관적으로 파악할 수 있습니다. 예를 들어, Topic 1의 대표 문서들이 ‘정부 정책’, ‘선거’와 같은 단어를 포함한다면, Topic 1을 ‘정치’ 토픽으로 명명할 수 있습니다.

#### 정의: ‘theta’ 행렬의 해석

STM 모델의 결과 객체에는 문서-토픽 분포를 나타내는 ‘theta’ 행렬이 포함됩니다. 이 행렬은 각 문서가 각 토픽에 얼마나 기여하는지를 확률 값으로 보여줍니다.

- **행:** 각 문서
- **열:** 각 토픽
- **값:** 문서가 해당 토픽에 기여하는 확률 (모든 토픽에 대한 확률의 합은 1)

예를 들어, ‘theta[1, 2] = 0.7’이라면 첫 번째 문서가 두 번째 토픽에 70% 기여한다는 의미입니다.

## 과제 및 제출 가이드라인

강의에서 언급된 과제 제출 관련 지침을 정리합니다.

### 과제 5: 텍스트 오토인코더

- **목표:** 텍스트 오토인코더의 구현 및 한계 이해. 완벽한 재현보다는 모델의 어려움을 경험하는 것이 중요합니다.
- **제출물:**
  - **보고서 (PDF):** Jupyter Notebook으로 생성하거나 수동으로 작성할 수 있습니다. 최종적으로는 **하나의 PDF 파일**로 통합하여 제출하는 것이 권장됩니다. 보고서에는 모델 구현, 결과, 그리고 결과가 기대만큼 좋지 않은 이유에 대한 논의가 포함되어야 합니다.
  - **코드 파일:** Jupyter Notebook(.ipynb) 파일 또는 기타 코드 파일. 여러 문제를 해결하기 위해 여러 Jupyter Notebook 파일을 제출하는 것이 허용됩니다.
  - **압축 파일 (ZIP):** 여러 코드 파일을 제출할 경우, 모든 파일을 하나의 ZIP 파일로 압축하여 제출하는 것이 좋습니다. 2 3개의 파일은 개별 제출도 허용되지만, 20개와 같이 파일 수가 많으면 ZIP 파일로 압축해야 합니다.
- **핵심:** 보고서와 원본 코드를 모두 제출하여, 보고서 내용을 검증할 수 있도록 해야 합니다.

### 과제 6: 구조적 토픽 모델링 (STM)

- **목표:** R 언어를 사용하여 STM 모델을 구축하고, 메타데이터의 영향을 분석하며, 최적의 토픽 수를 탐색하는 실습.
- **데이터:** 선택한 뉴스 기사 데이터셋 (강의에서 사용된 Kaggle 데이터셋 포함).
- **제출물:** 과제 5와 동일한 제출 가이드라인을 따릅니다. R 코드와 보고서를 포함해야 합니다.

#### 주의: 제출물 누락 방지

가장 중요한 것은 보고서와 원본 코드를 모두 제출하는 것입니다. 파일 압축 여부는 부차적인 문제이지만, 여러 파일일 경우 ZIP으로 묶는 것이 채점자에게 편리합니다. 보고서가 없거나 코드가 없으면 채점 자체가 불가능합니다.

## 체크리스트

학습 내용 복습, 과제 작성 및 제출 시 활용할 수 있는 체크리스트입니다.

### 학습 내용 점검

- 텍스트 오토인코더의 어려움과 이미지 오토인코더와의 차이점을 이해했는가?
- LDA의 기본 가정(문서-토픽 혼합)과 동작 원리를 이해했는가?
- LDA가 비지도 학습이며, 레이블 데이터가 필요 없음을 이해했는가?
- LDA에서 최적 토픽 수  $K$ 를 결정하는 어려움과 평가 지표(일관성, 배타성)를 이해했는가?
- LDA가 확률적 모델이며, 실행마다 결과가 달라질 수 있음을 이해했는가?
- NMF가 행렬 분해 기반의 토픽 모델임을 이해했는가?
- LDA와 NMF의 주요 차이점을 설명할 수 있는가?
- 최대 우도 추정(MLE)의 개념과 잠재 변수가 있을 때 EM 알고리즘이 사용되는 이유를 이해했는가?
- STM이 LDA의 한계를 어떻게 극복하고 메타데이터를 활용하는지 이해했는가?
- STM에서 토픽 비율( $\theta_d$ )이 로지스틱 정규 분포에서 생성되는 과정을 이해했는가?
- R 언어에서 'stm' 패키지를 설치하고 로드하는 방법을 아는가?
- 'textProcessor'와 'prepareDocuments' 함수를 사용하여 텍스트를 전처리하는 방법을 아는가?
- 'stm()' 함수로 모델을 훈련하고 'summary()', 'plot()'으로 결과를 해석하는 방법을 아는가?
- 'estimateEffect()' 함수로 공변량의 영향을 추정하고 시각화하는 방법을 아는가?
- 'searchK()' 함수로 최적 토픽 수  $K$ 를 탐색하고 평가 지표를 해석하는 방법을 아는가?
- 'findThoughts()' 함수로 토픽의 의미를 파악하는 방법을 아는가?

### 과제 제출 점검

- 과제 보고서(PDF)를 작성했는가?
- 보고서에 모든 문제에 대한 해결 과정, 결과, 분석 및 논의가 포함되어 있는가?
- 보고서가 단일 PDF 파일로 통합되어 있는가?
- 모든 원본 코드 파일(Jupyter Notebook 등)을 포함했는가?
- 여러 코드 파일이 있다면 하나의 ZIP 파일로 압축했는가?



- 제출 파일에 출처, 파일명, 페이지 언급 등 금지된 내용이 없는가?
- 모든 파일이 Canvas에 올바르게 업로드되었는가?

## FAQ (자주 묻는 질문)

**Q: 텍스트 오토인코더가 이미지 오토인코더보다 어려운 이유는 무엇인가요?**

**A:** 텍스트는 이산적인 단어들의 시퀀스로 구성되며, 단어의 순서와 문맥이 의미를 결정하는데 매우 중요합니다. 이미지는 연속적인 픽셀 값으로 공간적 인접성이 중요하죠. 텍스트를 단일 벡터로 압축하는 과정에서 이러한 시퀀스 정보(시간적 요소)가 크게 손실되는 '병목 현상'이 발생하여, 원본 문장을 완벽하게 복원하기가 매우 어렵습니다.

**Q: LDA는 왜 실행할 때마다 결과가 달라질 수 있나요?**

**A:** LDA는 확률론적 모델이며, EM 알고리즘과 같은 반복적인 최적화 방법을 사용합니다. 이 과정에서 우도 함수가 여러 지역 최대값(local maxima)을 가질 수 있어, 알고리즘의 초기값이나 내부적인 확률적 샘플링에 따라 다른 결과에 수렴할 수 있습니다. 따라서 여러 번 실행하여 가장 좋은 결과를 선택하는 것이 일반적입니다.

**Q: STM에서 메타데이터는 어떻게 토픽 모델링에 활용되나요?**

**A:** STM은 LDA의 토픽 생성 과정 중, 문서별 토픽 분포( $\theta_d$ )를 생성하는 부분에 메타데이터를 통합합니다. 구체적으로,  $\theta_d$ 가 로지스틱 정규 분포에서 생성되는데, 이 분포의 중심(평균)이 메타데이터( $X_d$ )와 계수( $\gamma$ )의 선형 결합( $X_d\gamma$ )으로 정의됩니다. 이를 통해 메타데이터 변수가 토픽의 출현 빈도(prevalence)에 미치는 영향을 모델링할 수 있습니다.

**Q: R Markdown(RMD)은 Jupyter Notebook과 어떻게 다른가요?**

**A:** 둘 다 코드, 텍스트, 실행 결과를 통합하는 도구이지만, RMD는 주로 R 언어에 최적화되어 있으며, LaTeX, HTML, Word 등 다양한 형식의 보고서 생성이 용이합니다. 코드 블록은 백틱(‘ ‘)으로 구분하며, RMD 파일 자체는 텍스트 기반이어서 버전 관리에 유리합니다. Jupyter Notebook은 Python을 포함한 다양한 언어를 지원하며, 웹 기반 인터페이스가 특징입니다.

**Q: ‘searchK0’로 최적 토픽 수  $K$ 를 찾을 때 어떤 지표를 봐야 하나요?**

**A:** 주로 의미론적 일관성(Semantic Coherence)과 배타성(Exclusivity)을 함께 고려합니다. 일관성은 토픽 내 단어들의 의미적 관련성을, 배타성은 토픽들이 서로 겹치지 않는 정도를 나타냅니다. 이 두 지표를 동시에 최대화하는  $K$ 를 선택하는 것이 이상적입니다. 시각화된 그래프에서 우측 상단에 위치하는  $K$ 를 선택하는 경향이 있습니다.

## 빠르게 훑어보기 (1페이지 요약)

### 핵심 요약: CSCI89 Lecture 8 핵심 요약

- **텍스트 오토인코더의 어려움:** 이미지와 달리 텍스트는 시퀀스 정보 손실이 커 완벽한 재현이 어렵습니다. 병목 현상, 관측치 부족이 주원인입니다. 데이터 증강이나 Softmax 온도 함수 활용, 또는 병목 없는 시퀀스-투-시퀀스 모델로 임베딩을 훈련하는 방법을 고려할 수 있습니다.
- **LDA (잠재 디리클레 할당):** 문서가 여러 토픽의 혼합으로 구성된다는 비지도 학습 모델입니다. 최적 토픽 수  $K$  결정이 어렵고, 확률적 특성으로 인해 실행마다 결과가 달라질 수 있습니다.
- **NMF (음수 행렬 분해):** 텍스트 행렬을 두 개의 비음수 행렬로 분해하여 잠재 토픽 구조를 찾는 행렬 대수 기반 모델입니다. LDA와 달리 결정론적이며, 차원 축소에 활용됩니다.
- **EM 알고리즘:** 잠재 변수(예: LDA의 토픽 할당)가 있는 모델의 파라미터를 추정하는 반복적인 최적화 방법입니다. 기대(E) 단계와 최대화(M) 단계로 구성됩니다.
- **STM (구조적 토픽 모델):** LDA를 확장하여 문서의 메타데이터(예: 저자, 출판일, 성별)를 토픽 모델링에 통합합니다. 메타데이터가 토픽 분포에 미치는 영향을 분석하고 가설을 검정할 수 있어 더 정확하고 해석 가능한 토픽을 제공합니다.
- **R 실습 ('stm' 패키지):**
  - **데이터 준비:** 뉴스 헤드라인 데이터셋을 로드하고, 'publish<sub>date</sub>' .
  - **텍스트 전처리:** 'textProcessor'와 'prepareDocuments'를 사용하여 불용어 제거, 어간 추출, 빈 문서 및 희귀 단어 제거 등을 수행합니다.
  - **모델 훈련 및 해석:** 'stm()' 함수로 모델을 훈련하고, 'summary()', 'plot()'으로 토픽별 단어 및 prevalence를 확인합니다.
  - **효과 분석:** 'estimateEffect()'로 공변량(예: 시간)이 토픽 prevalence에 미치는 영향을 추정하고 시각화합니다.
  - **최적 K 탐색:** 'searchK()'로 다양한 토픽 수  $K$ 에 대해 모델을 실행하고, 의미론적 일관성 및 배타성 지표를 비교하여 최적  $K$ 를 선택합니다.
  - **토픽 이해:** 'findThoughts()'로 각 토픽과 가장 연관된 문서를 확인하여 토픽의 의미를 파악합니다.
- **과제 제출:** 보고서(PDF)와 모든 코드 파일(ZIP 압축 권장)을 함께 제출해야 합니다.